

Git Branches & Pull Request - Class Notes and Practice Assignment

Git Branches & Pull Request - Class Notes

1. What We Did in Class

Today we learned how to create a Git project, push it to GitHub, create a new branch, push that branch, raise a Pull Request, merge it on GitHub, and finally pull the latest changes back into the local system.

Main flow:

```
Local project
→ push to main
→ create feature branch
→ modify file
→ commit on feature branch
→ push feature branch
→ create Pull Request on GitHub
→ merge Pull Request into main
→ pull latest main into local system
```

Important memory line:

```
Branch first. Work safely. Merge through Pull Request.
```

2. Why Branches Are Needed

In real projects, developers should not directly experiment on the `main` branch.

`main` should contain stable code.

A branch is a separate line of work where we can safely make changes without disturbing `main`.

Example:

```
main = stable project
ui-updates = branch for UI changes
login-feature = branch for login work
footer-update = branch for footer changes
```

Simple analogy:

```
main = final notebook
branch = rough notebook
Pull Request = asking someone to check your work
merge = approved work added back to main
```

3. Commands Practiced in Class

Create Project Folder

```
mkdir git-branch
cd git-branch
code .
```

Meaning:

```
mkdir git-branch = create a folder named git-branch
cd git-branch = enter the folder
code . = open current folder in VS Code
```

4. Create `index.html`

Inside the `git-branch` folder, create a file:

```
index.html
```

Add some basic random HTML content.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Git Branch Practice</title>
</head>
<body>
  <h1>Git Branch Practice</h1>
  <p>This file was created on the main branch.</p>
</body>
</html>
```

5. Push First Version to **main**

Run these commands:

```
git init
git add .
git commit -m "A"
git remote add origin <repo-url>
git branch -M main
git push -u origin main
```

Explanation:

```
git init = start Git tracking
git add . = move all files to staging area
git commit -m "A" = save first checkpoint locally
git remote add origin <repo-url> = connect local project to GitHub repository
```

```
git branch -M main = name the main branch as main  
git push -u origin main = push local main branch to GitHub
```

Important:

```
Commit is local.  
Push is online.
```

6. Create a New Branch

Create and switch to a new branch named `ui-updates`:

```
git switch -c ui-updates
```

Meaning:

```
git switch = move to another branch  
-c = create a new branch  
ui-updates = branch name
```

Full meaning:

```
Create a branch named ui-updates and move into it.
```

7. Modify File in Branch

Open `index.html` and add one more line:

```
<p>This line was added from the ui-updates branch.</p>
```

Save the file.

Now commit this branch change:

```
git add .  
git commit -m "B"
```

Explanation:

This commit is created on the ui-updates branch, not directly on main.

8. Push Branch to GitHub

Push the `ui-updates` branch to GitHub:

```
git push -u origin ui-updates
```

Meaning:

Upload the ui-updates branch to GitHub.

After this, GitHub can see both branches:

```
main  
ui-updates
```

9. Create Pull Request on GitHub

After pushing `ui-updates`, open GitHub.

GitHub may show:

Compare & pull request

Click it.

Pull Request meaning:

```
I completed work in my branch.  
Please review and merge it into main.
```

In the Pull Request screen:

```
base branch = main  
compare branch = ui-updates
```

Meaning:

```
ui-updates changes will be merged into main.
```

Visual:

```
ui-updates —————> main
```

10. Merge Pull Request

On GitHub, click:

```
Merge Pull Request  
Confirm Merge
```

After this:

```
GitHub main branch now contains ui-updates changes.
```

But local `main` is still old.

So we need to pull latest changes locally.

11. Pull Latest Main Locally

Come back to local terminal.

Switch to main:

```
git switch main
```

Pull latest changes from GitHub main:

```
git pull origin main
```

Meaning:

```
Bring latest GitHub main changes into local main branch.
```

Important line:

```
After merging PR on GitHub, local main must pull latest changes.
```

12. Complete Flow Summary

```
mkdir git-branch  
cd git-branch  
code .  
  
# create index.html  
  
git init  
git add .  
git commit -m "A"  
git remote add origin <repo-url>  
git branch -M main
```

```
git push -u origin main

git switch -c ui-updates
# modify index.html

git add .
git commit -m "B"
git push -u origin ui-updates

# create PR on GitHub
# merge PR into main on GitHub

git switch main
git pull origin main
```

13. Important Concepts

Branch

A branch is a separate place to work safely without directly disturbing main.

Main Branch

main is the stable version of the project.

Feature Branch

A feature branch is created for a specific task or update.

Example:


```
ui-updates  
feature-login  
feature-footer
```

Pull Request

A Pull Request is a request to merge one branch into another branch on GitHub.

Base Branch

The branch that receives the changes.
Example: main

Compare Branch

The branch that contains the new changes.
Example: ui-updates

Pull After PR Merge

If PR is merged on GitHub, local main will not automatically update.
Use `git pull origin main`.

14. Simple Practice Assignment

Task Name

Practice Git Branch and Pull Request Flow

Goal

Create a new branch, make one small change, push the branch, create a Pull Request, merge it, and pull latest main locally.

Assignment Steps

Step 1: Use Existing Repository

Use the same repository created in class:

```
git-branch
```

Open terminal inside the project folder.

Check current branch:

```
git branch
```

Switch to main:

```
git switch main
```

Pull latest main:

```
git pull origin main
```

Step 2: Create New Branch

Create a new branch:

```
git switch -c content-update
```

Step 3: Modify `index.html`

Open `index.html` and add this line before `</body>`:

```
<p>This line was added from content-update branch.</p>
```

Save the file.

Step 4: Commit the Change

```
git status
git add .
git commit -m "added content update"
```

Step 5: Push Branch to GitHub

```
git push -u origin content-update
```

Step 6: Create Pull Request

Go to GitHub.

Create a Pull Request:

```
base: main
compare: content-update
```

Then click:

```
Create Pull Request
Merge Pull Request
Confirm Merge
```

Step 7: Pull Latest Main Locally

After merging PR on GitHub, come back to terminal.

```
git switch main  
git pull origin main
```

Open `index.html` and confirm that the new line is visible in local main.
